WEB TECHNOLOGIES 2026 · BIUH · LESSON 4

Structure, Navigation and Not Repeating Yourself

Favicon · page anatomy · a third page · DRY · your first JavaScript

webtech.landrock.io

WHERE YOU ARE

You have two pages.

`index.html`	your first page, written by hand
`quotes.html`	the exercise — five quotes, or scaffolding for them

Today they become a site: three pages, one identity, one navigation — written once.

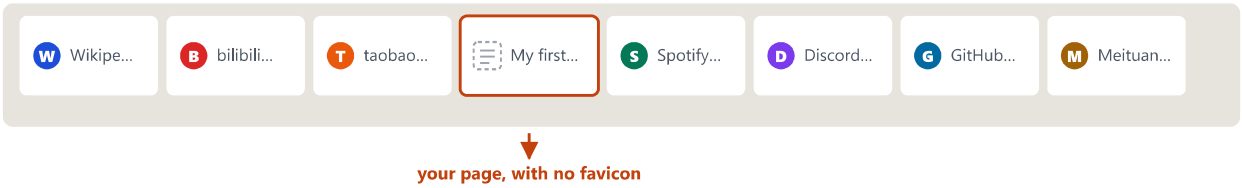
FIRST, SOMETHING SMALL

The favicon

The little picture in the browser tab

WHY BOTHER WITH A 32-PIXEL IMAGE?

It is how people find you again.



Open fifteen tabs and the titles shrink to nothing. The icon is all that is left.

It also appears in bookmarks, in history, and on a phone's home screen.

ONE RULE BEFORE YOU MAKE ONE

It must be square.

32 × 32. 180 × 180. 512 × 512. Always the same number twice.

Give the browser a rectangle and it will squash it to fit — your logo comes out stretched, and you will not notice until someone shows you.

Design on a square canvas from the start. Cropping a wide logo down to a square is a design decision, not an export setting.

WHICH FILE, AND HOW BIG

PNG is all you need today.

FORMAT	USE IT?	WHY
PNG	yes — start here	works everywhere, supports transparency, easy to export from any tool
SVG	later	scales perfectly and can change in dark mode — but you have to be able to make one
.ico	not needed	the old Windows format. Only for very old browsers
JPEG	no	no transparency, and it smudges at small sizes

We will come back to SVG once you can produce one. PNG today.

SIZES — AND WHY ONE IS NOT ENOUGH

Two files cover almost everything.

FILE	SIZE	WHERE IT IS USED
`favicon-32.png`	32 × 32	the browser tab, bookmarks, history
`apple-touch-icon.png`	180 × 180	a phone's home screen when someone saves your site
`favicon-192.png`	192 × 192	Android home screen — optional, same picture

Phones have high-resolution screens. A 32-pixel image blown up to home-screen size looks like a mistake — so the large one is a separate, properly sized file, not the small one stretched.

Design it once at 512 × 512, then export down. Never scale a small image up.

THREE LINES IN THE HEAD

```
<link rel="icon" href="favicon-32.png" sizes="32x32">
<link rel="icon" href="favicon-192.png" sizes="192x192">
<link rel="apple-touch-icon" href="apple-touch-icon.png">
```

The browser picks the one it needs. You are not choosing for it — you are offering it options.

All three go in , next to the . Same job: telling a machine what this page is called and what it looks like. `<head>` `<title>`

Make it from a letter, a shape, a symbol. Thirty-two pixels is not a canvas — it is a signature.

RECAP · FROM LESSON 1

You just chose a file format.
Here is the rest of the set.

A favicon is a PNG. But every image, font and video on your site is a choice like that one — and each has a right answer.

CHOOSING A FORMAT IS A REAL DECISION

The families, and the rule for each.

FOR…	USE	THE RULE
Photographs	`WebP`	Far smaller than JPEG at the same quality. Never ship a camera original.
Logos, icons	`SVG`	It is text. Scales to any size. CSS can style it.
Transparency	`PNG / WebP`	PNG works and is large. WebP does it smaller.
Animation	`MP4`	**Not GIF.** Ten times the size for worse quality.
Fonts	`WOFF2`	Every font is a request — and text is invisible until it arrives.

ONE PHOTO FROM A PHONE · THE SAME PICTURE EVERY TIME

Same image. Four decisions.

original	4.0 MB
JPEG q80	850 KB
WebP	480 KB
AVIF	320 KB
+ resized	85 KB

The biggest win is not the format. It is not sending a 4000px image into a 600px box.

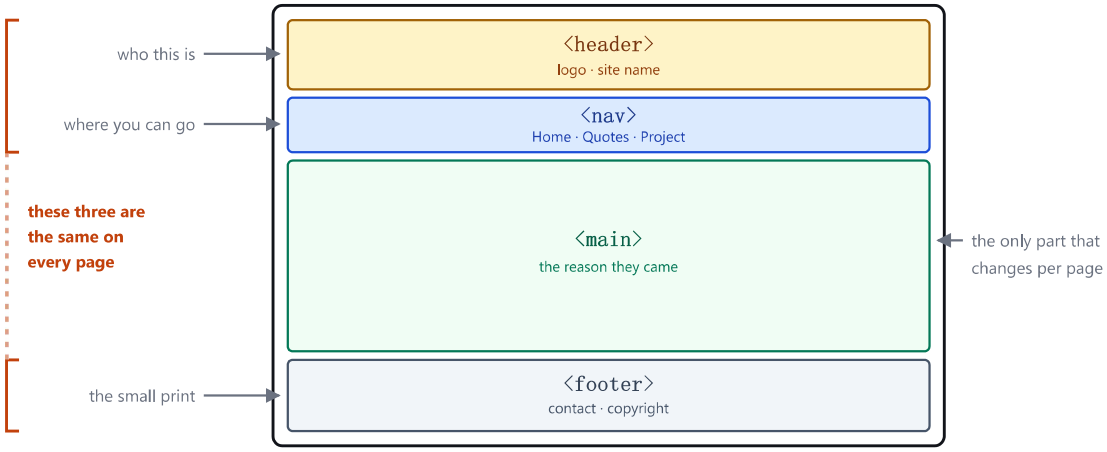
Numbers are illustrative — measure the real ones on your own photo.

BEFORE YOU BUILD THE THIRD PAGE

What a page is made of

NEARLY EVERY PAGE YOU HAVE EVER USED

The same four parts — learn the names.



Sketch this before you write markup. It takes a minute and decides the whole page — and three of the four parts you will write once and reuse.

IN HTML, THOSE FOUR PARTS HAVE NAMES

```
<html>
<head>
  <!-- nobody sees this: title, charset, favicon, CSS -->
</head>

<body>
  <header>
    <nav> … </nav>
  </header>

  <main>
    <!-- the only part that differs per page -->
  </main>

  <footer> … </footer>
</body>
</html>
```

You met these in lesson 2. Now they have a job: a shape you can reuse.

TWO WORDS, FOUR LETTERS APART, COMPLETELY DIFFERENT JOBS

<head> is not .<header>

<head>

Nobody sees it. It sits above and holds information about the page. `<body>`

The title in the tab, the charset, the favicon, the link to your CSS. Machines read all of it.

One per page.

<header>

Everybody sees it. It sits inside and is the top of what is on screen. `<body>`

Your site name, your logo, usually the navigation.

You can have **several** — one per page, and one inside each article if you want.

head = about the page. header = top of the page.

The same trap exists at the bottom: <footer> is visible content, and there is no <foot>.

NOW BUILD IT

project.html

Your third page — and the first one that is about your work

THIS PAGE IS FOR THINKING, NOT POLISH

What could your project be?

Write down **five ideas**. Not one — five. The first idea is rarely the best one, and you cannot compare a list of one.

```
<main>
  <h1>Project ideas</h1>

  <h2>Things I might build</h2>
  <ul>
    <li>A site about the food in my hometown</li>
    <li>A portfolio for my photographs</li>
    <li>…</li>
  </ul>
</main>
```

That <ul> is a list. There are three kinds and the choice carries meaning — next slide.

TWO KINDS OF LIST, AND THE DIFFERENCE IS MEANING

 and <ol>.

`<ul>`
unordered **l**ist — the order carries no meaning

`<ol>`
ordered **l**ist — the order is the information

`<li>`
list **i**tem — one entry, used by both

Ask yourself: if I shuffled these, would anything be lost? No → . Yes → . `<ul>` `<ol>`

Your project ideas are a <ul> — you have not ranked them. Ingredients are a <ul>; the steps to cook them are an <ol>.

THE MARKUP, AND WHAT THE BROWSER DOES WITH IT

You write

```
<ul>
  <li>Hometown food</li>
  <li>Photo portfolio</li>
</ul>

<ol>
  <li>Sketch it</li>
  <li>Write the HTML</li>
  <li>Style it</li>
</ol>
```

The browser shows

- Hometown food
- Photo portfolio

1. Sketch it
2. Write the HTML
3. Style it

You never type the bullets or the numbers. The browser adds them, because you said what kind of list it is.

Which means you can never get the numbering wrong — delete item 2 and the rest renumber themselves.

FOR EACH IDEA, ONE QUESTION

Who is your project — your website — for?

"Everyone" is not an answer. Name a person you could actually show it to.

So each idea is now a name and a description. There is a list type built for exactly that pairing.

A THIRD KIND OF LIST

<dl> — the description list.

<dl>
description **l**ist — the whole list

<dt>
description **t**erm — the thing being named

<dd>
description **d**etail — what it means

Use it wherever content comes in pairs: a term and its meaning, a question and its answer, a name and its explanation.

A glossary is a <dl>. So is a FAQ. So is a list of project ideas with descriptions.

THE MARKUP, AND WHAT THE BROWSER DOES WITH IT

You write

```
<dl>
  <dt>Hometown food</dt>
  <dd>For students who just
      moved here.</dd>

  <dt>Photo portfolio</dt>
  <dd>For places I want to
      apply to.</dd>
</dl>
```

The browser shows

Hometown food

For students who just moved here.

Photo portfolio

For places I want to apply to.

The is indented — the browser is showing you it belongs to the term above it. `<dd>`

No CSS here. That indent is the default styling, and it is telling you the two are related.

NOW PUT A NAVIGATION ON ALL THREE PAGES

Copy it. By hand. Three times.

```
<nav>
  <a href="index.html">Home</a>
  <a href="quotes.html">Quotes</a>
  <a href="project.html">Project</a>
</nav>
```

Yes, really. Copy and paste it into each page.

I am asking you to do something I am about to tell you not to do. That is deliberate.

NOW ADD A FOURTH PAGE



Three files to edit.

Miss one and your site is quietly broken.

With twenty pages it is twenty edits — and you will miss one.

THIS PROBLEM HAS A NAME — AND A FIX

DRY — Don't Repeat Yourself.

> "*Every piece of knowledge must have a single, unambiguous, authoritative representation within a system.*"
>
> — Andy Hunt and Dave Thomas, **The Pragmatic Programmer**, 1999

Read it again: knowledge, not code. The navigation is knowledge — "these are the pages of this site". It should exist in **one place**.

DRY is not about typing less. It is about having one place to change something when it changes — and it always changes.

THE FIX

Your first JavaScript

One navigation. Loaded into every page.

STEP ONE

Create a new file: `nav.html`

Same folder as your other pages. Put only the navigation in it:

```
<nav>
  <a href="index.html">Home</a>
  <a href="quotes.html">Quotes</a>
  <a href="project.html">Project</a>
</nav>
```

One file. One place to change it. That is DRY — and so far there is no JavaScript at all.

SOMETHING WILL LOOK WRONG ABOUT THAT FILE

No . No . No or .<!DOCTYPE><html><head><body>

That is deliberate. is not a page — it is a fragment. A piece that gets inserted into a page that already has all of that. `nav.html`

If you gave it the full skeleton, you would end up with an inside an — which is not valid, and the browser would quietly throw most of it away. `<html>` `<html>`

Do not open nav.html in your browser.
On its own it is not meant to work.

The .html extension is a convention here — it tells your editor to highlight the tags. Some people name these files .part.html or .inc.html to make the difference obvious.

STEP TWO

In each of your three pages.

```
<!-- index.html, quotes.html, project.html -->
<header id="site-header">
  <!-- the nav used to be here. Now it is empty. -->
</header>

<!-- before </body> -->
<script src="nav.js"></script>
```

Delete the copied-out and give the you already have an **id**, so the script can find it.
`<nav>` `<header>`

The script goes at the end of the body so the header exists before the code runs. There are other ways; this one is simplest.

STEP THREE · SIX LINES OF JAVASCRIPT

```
// nav.js
fetch('nav.html')
  .then(response => response.text())
  .then(html => {
    document.getElementById('site-header').innerHTML = html;
  });
```

fetch — go and get that file. **.text()** — read it as text. **innerHTML** — put it inside the header.

You will meet all three again properly. Today you only need to know what they do.

ONE THING WILL GO WRONG — AND IT IS THE USEFUL PART

Open it with and it fails.`file://`

The page loads. The navigation does not. The console says something about CORS or cross-origin request blocked.

fetch needs a server.

This is not a bug in your code and not something you can switch off. It is a browser security rule: a page opened from your disk is not allowed to go reading other files from your disk.

Imagine if it could. A page you opened from an email could read anything in your home folder.

THE FIX — AND IT TAKES ONE MINUTE

Install Live Server in VS Code.

1. Open the **Extensions** panel in VS Code (the squares icon in the left bar)
2. Search for **Live Server** and click **Install**
3. Right-click → **Open with Live Server** `index.html`

Your browser opens at — and the navigation appears. `http://127.0.0.1:5500`

No terminal, nothing else to install. It also reloads the page every time you save, which you will not want to work without again.

Look at the address bar: http:// — not file://. That is the whole difference.

WHAT YOU JUST DID

That is a component.

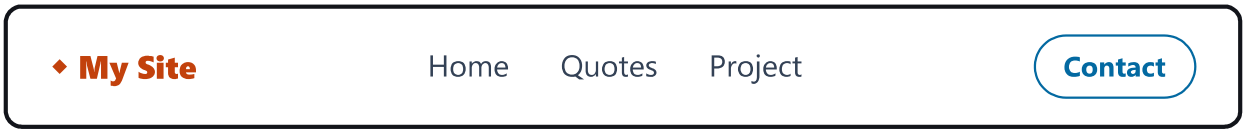
One piece of the page, defined once, used in many places. Change and every page updates. `nav.html`

Frameworks — React, Vue — are largely this idea, made faster and safer. You now know what problem they were invented to solve.

Six lines of JavaScript. Not a framework — and not needed yet.

WHAT GOES IN THAT HEADER

Three things, one row.



logo or site name — left *navigation — centred* *one action — right*

Nearly every site you use has this shape. Left, centre, right — identity, where you can go, and the one thing they want you to do.

The third slot is optional. Leave it empty rather than inventing something to put there.

THE MARKUP — NAV.HTML

```
<nav>
  <a class="logo" href="index.html">◆ My Site</a>

  <ul class="links">
    <li><a href="index.html">Home</a></li>
    <li><a href="quotes.html">Quotes</a></li>
    <li><a href="project.html">Project</a></li>
  </ul>

  <a class="action" href="mailto:me@example.com">Contact</a>
</nav>
```

A list of links is a list — so , as you learned earlier. The logo and the action are single links, so they are not in it. `<ul>`

Three children of <nav>: logo, list, action. That is what the CSS will position.

THE CSS — ONE LINE DOES THE WORK

```
nav {
  display: flex;
  align-items: center;      /* vertically centred */
  gap: 1rem;
  padding: 1rem 2rem;
}

nav .links {
  display: flex;
  gap: 1.5rem;
  margin: 0 auto;           /* pushes it to the centre */
  list-style: none;         /* no bullets */
  padding: 0;
}
```

`display: flex` puts the three children in a row. on the middle one takes all the leftover space on both sides — which centres it and pushes the other two apart. `margin: 0 auto`

That is the layout done. The three pieces are in position — now they need to look like something.

THE CSS — HOW THE THREE PIECES LOOK

```
nav a {
  text-decoration: none;      /* no underlines */
  color: #334155;
}

nav .logo {
  font-weight: 800;
  color: #c2410c;
}

nav .action {
  border: 1.5px solid #0369a1;
  border-radius: 99px;        /* a pill */
  padding: .3em 1em;
  color: #0369a1;
}

nav a:hover { color: #c2410c; }
```

`border-radius: 99px` is the pill trick — any number larger than half the height gives you a fully rounded end, so you never have to calculate it.

The action looks like a button but it is an <a>. It goes somewhere, so it is a link — appearance does not change what an element is.

ONE MORE PIECE

Now the footer.

The same problem, and you already know the shape of the answer

STEP ONE · A SECOND FRAGMENT

Create `footer.html`

```
<p>Built by Li Wei · Web Technologies 2026</p>

<address>
  <a href="mailto:liwei@example.com">liwei@example.com</a>
</address>
```

No tag inside it — the page already has one, exactly as had no .
`<footer>` `nav.html` `<header>`

`<address>` means contact details for this page or article. It is one of the few places that element is correct — not for any postal address that happens to appear in your text.

STEP TWO · GIVE THE FOOTER A NAME TOO

```
<!-- in all three pages -->
<footer id="site-footer">
</footer>
```

Identical to what you did with : the element is already there and already correct. It just needs an . `<header>` `id`

Two empty elements, two fragments, one script.

STEP THREE · YOU COULD WRITE FETCH TWICE…

```
fetch('nav.html')
  .then(r => r.text())
  .then(html => {
    document.getElementById('site-header').innerHTML = html;
  });

fetch('footer.html')
  .then(r => r.text())
  .then(html => {
    document.getElementById('site-footer').innerHTML = html;
  });
```

Look at it. You have just repeated yourself.

Two blocks, identical apart from two words.

STEP THREE · SO WRITE IT ONCE AND CALL IT TWICE

```
// parts.js
function load(file, target) {
  fetch(file)
    .then(r => r.text())
    .then(html => {
      document.getElementById(target).innerHTML = html;
    });
}

load('nav.html', 'site-header');
load('footer.html', 'site-footer');
```

A **function** is a piece of work you describe once and then run with different values. and are the parts that change. `file target`

Rename the file: it is not nav.js any more. Call it parts.js or include.js — and update the <script src> in all three pages.

THAT IS DRY AGAIN — THIS TIME IN YOUR JAVASCRIPT

You applied it to your own code.

First you stopped repeating HTML across three pages. Then you stopped repeating JavaScript inside one file.

Add a fourth page now, and you edit one file.

The nav, the footer and the script are all in one place each. That is the whole point of the last hour.

LESSON 4 · DONE

What you should be able to say now.

- Why a favicon is worth three lines in the head — and why it must be square
- The four parts nearly every page is made of, and that is not `<head>` `<header>`
- Which list to reach for: , or `<ul>` `<ol>` `<dl>`
- What DRY actually protects you from
- What does — and why it needs a server `fetch`
- How to lay out a header: logo left, nav centred, action right
- What a function is for — write it once, call it with different values

`webtech.landrock.io/sessions`

Next: twenty quotes, and a layout worth looking at.